

Arquiteturas de Sistemas Distribuídos, Middleware Amazon EventBridge e Análise Comparativa

Andrio Epping, Gabriel Silveira Marques e Bruna Letícia Schneiders

31/08/2026

Resumo

Este relatório apresenta uma análise dos princípios que fundamentam os sistemas distribuídos, com ênfase no papel do *middleware* como ferramenta de mitigação da heterogeneidade física e lógica entre componentes de software. Partindo de um percurso histórico e técnico, o trabalho descreve a transição de integrações fortemente acopladas — baseadas em comunicações síncronas — para arquiteturas orientadas a eventos e mensagens. Realiza-se, também, uma comparação entre o estilo arquitetural REST e a Arquitetura Orientada a Eventos (EDA), considerando métricas de modularidade, coesão, acoplamento e complexidade estrutural. Por fim, o documento apresenta um *benchmarking* das principais tecnologias de *middleware* corporativo disponíveis no mercado, analisando latência, vazão e resiliência em cenários de carga controlada.

Sumário

1	Introdução	3
2	Middleware em Sistemas Distribuídos	4
2.1	Tipos e Evolução do Middleware	5
2.2	As Dimensões do Desacoplamento	7

2.3	Integração Corporativa e Padrões Arquiteturais	8
3	Comparações de Arquiteturas e Soluções de Mensageria	9
3.1	Arquitetura Orientada a Eventos (EDA) <i>vs.</i> Estilo REST	10
3.1.1	Separação de Interesses e Acoplamento	10
3.1.2	Coesão, Complexidade e Tamanho do Código	11
3.2	Comparativo Multidimensional de Plataformas de Middleware	12
3.2.1	Análise dos Resultados de Desempenho	13
3.2.2	O <i>Trade-off</i> Central	15
4	Conclusão	15

1 Introdução

O estudo de sistemas distribuídos ocupa uma posição central na engenharia de software contemporânea. A definição clássica, proposta por Tanenbaum e Van Steen [1], estabelece que um sistema distribuído é uma coleção de computadores independentes que se apresenta ao usuário como um sistema único e coerente. Essa definição aponta para dois aspectos que merecem atenção: primeiro, o hardware é composto por máquinas autônomas, cada uma com seus próprios recursos de processamento e memória; segundo, o software que orquestra essas máquinas deve prover uma abstração unificada, de modo que a separação física entre os nós não seja perceptível ao usuário final. Na prática, essa ideia é menos trivial do que parece, porque manter essa ilusão de sistema único exige resolver problemas de comunicação, consistência e tolerância a falhas que simplesmente não existem em uma aplicação monolítica rodando em uma única máquina.

A consolidação do paradigma distribuído tem raízes na necessidade concreta de escalar aplicações para além dos limites de processamento de infraestruturas centralizadas. A partir dos anos 2000, e com maior intensidade após a popularização da computação em nuvem, as organizações passaram a fragmentar suas aplicações em serviços menores e independentes. Essa migração, que Coulouris et al. [2] descrevem como um reflexo natural da evolução das demandas computacionais, não foi um movimento puramente estético: ela respondeu a gargalos reais de desempenho, disponibilidade e manutenibilidade que os monólitos centralizados já não conseguiam resolver de forma satisfatória.

Historicamente, as primeiras gerações de software distribuído apoiaram-se em primitivas de comunicação síncrona. O *Remote Procedure Call* (RPC) e, posteriormente, as requisições HTTP estruturadas segundo o estilo REST (*Representational State Transfer*) dominaram as integrações entre serviços. Esse modelo funcionava razoavelmente bem enquanto os sistemas tinham poucos componentes e a latência de rede era tolerável. O problema emergiu quando a complexidade cresceu: em chamadas síncronas, o serviço de origem precisa aguardar a resposta do serviço de destino antes de prosseguir. Quando um desses serviços fica lento ou indisponível,

o efeito não se limita ao ponto de falha — ele se propaga em cascata pela cadeia de dependências. Threads são bloqueadas, recursos computacionais se esgotam, e o que começou como uma degradação pontual rapidamente se transforma em uma indisponibilidade sistêmica. A literatura de engenharia de software descreve esse fenômeno como o surgimento do “monólito distribuído”: uma arquitetura que tem a complexidade operacional dos microsserviços, mas herdou o acoplamento rígido do monólito que pretendia substituir.

Para contornar essas limitações, os sistemas modernos migraram progressivamente para modelos de comunicação indireta e assíncrona. Em vez de um serviço chamar outro diretamente e esperar a resposta, a mensagem é depositada em um intermediário — o *middleware* — que se encarrega de entregá-la ao destinatário quando este estiver disponível. Essa mudança exigiu a evolução de uma das camadas mais sensíveis da arquitetura distribuída. A introdução de barramentos de mensagens e plataformas orientadas a eventos permitiu não apenas um ganho de vazão no tráfego de dados, mas garantiu que falhas operacionais ficassem confinadas ao componente afetado, sem derrubar o restante do ecossistema.

Este relatório tem três objetivos. O primeiro é apresentar os conceitos de *middleware* aplicados a sistemas distribuídos, explicando as funções e os mecanismos que essas plataformas oferecem. O segundo é comparar, com base em dados empíricos, as arquiteturas baseadas em requisição-resposta (REST) com aquelas baseadas na orientação a eventos (EDA), avaliando as consequências de cada abordagem sobre a modularidade e a complexidade do código. O terceiro é realizar um *benchmarking* das principais tecnologias de *middleware* disponíveis no mercado, contrastando seus perfis de latência, vazão e custo operacional.

2 Middleware em Sistemas Distribuídos

O *middleware* pode ser definido, em termos estruturais, como uma camada de software posicionada entre o sistema operacional e as aplicações distribuídas que rodam nos diversos nós da rede. Sua função central é abstrair a heterogeneidade inerente a esses ambientes: diferenças de plataforma de execução, sistema operaci-

onal, linguagem de programação e protocolo de comunicação. Na ausência de um *middleware*, cada aplicação precisaria lidar diretamente com essas incompatibilidades, o que multiplicaria o esforço de desenvolvimento e tornaria o sistema frágil a qualquer mudança na infraestrutura subjacente.

Para entender por que essa camada intermediária se tornou tão relevante, basta pensar em um cenário corporativo típico. Uma empresa de médio porte pode ter um sistema de vendas escrito em Java, um módulo de logística em Python, um serviço de pagamentos que expõe uma API REST e um sistema legado de contabilidade que ainda depende de protocolos proprietários. Fazer esses componentes conversarem entre si sem uma camada de abstração exigiria que cada equipe de desenvolvimento conhecesse os detalhes internos dos outros sistemas — exatamente o tipo de acoplamento que a engenharia de software tenta evitar. O *middleware* resolve esse problema ao oferecer uma interface uniforme de comunicação, independente das particularidades de cada componente.

2.1 Tipos e Evolução do Middleware

A trajetória do *middleware* reflete, de certa forma, a própria evolução das necessidades corporativas em lidar com dados distribuídos. As primeiras implementações eram fortemente acopladas e tinham como propósito principal abstrair a rede em si. Dois exemplos emblemáticos dessa fase inicial são a arquitetura CORBA (*Common Object Request Broker Architecture*) e as implementações de RPC. Em ambos os casos, a premissa era permitir que um programa em uma máquina chamasse procedimentos ou métodos em outra máquina de maneira transparente, como se fossem chamadas locais. Funcionava, mas com restrições significativas: a comunicação era síncrona, o acoplamento entre emissor e receptor era forte, e a escalabilidade era limitada pela capacidade do servidor que hospedava o objeto remoto.

Com o tempo, as limitações desses modelos levaram ao surgimento de dois paradigmas de comunicação assíncrona que hoje dominam o cenário do *middleware* corporativo:

- **MOM (*Message-Oriented Middleware*):** Gerencia a comunicação entre

componentes através do transporte e do roteamento de mensagens formatadas. A estrutura mais comum no MOM é a fila ponto a ponto (*point-to-point*): um produtor deposita uma mensagem em uma fila física, e um consumidor retira essa mensagem para processá-la. A fila funciona como um amortecedor temporal — se o consumidor estiver indisponível no momento em que a mensagem chega, ela permanece armazenada até que alguém a consuma. Esse mecanismo garante persistência e ordenação. Um exemplo concreto desse paradigma é o Amazon SQS, que implementa filas gerenciadas na nuvem com retenção de até 14 dias.

- **EOM (*Event-Oriented Middleware*):** Opera sob uma lógica diferente. Em vez de trabalhar com mensagens endereçadas a um consumidor específico, o EOM propaga mudanças de estado — chamadas de eventos — para múltiplos interessados simultaneamente. O modelo mais comum é o publicador-assinante (*publish-subscribe*): um componente publica um evento em um barramento ou tópico, e todos os assinantes registrados naquele tópico recebem a notificação de forma autônoma. Os consumidores não precisam saber quem publicou o evento, e o publicador não precisa saber quantos consumidores existem. Essa independência entre as partes é o que a literatura chama de desacoplamento. O Amazon EventBridge [8] e o Google Cloud Pub/Sub são exemplos de plataformas que implementam esse modelo em escala de nuvem.

Na prática, a fronteira entre MOM e EOM nem sempre é nítida. Tecnologias como o Apache Kafka [7], por exemplo, combinam características de ambos os paradigmas: utilizam tópicos com múltiplos assinantes (semelhante ao EOM), mas armazenam as mensagens em um log persistente e imutável que os consumidores consultam por operações de *pull* (uma característica associada ao MOM). Essa hibridização reflete uma tendência do mercado de convergir para soluções que ofereçam flexibilidade suficiente para atender tanto a cenários de fila quanto a cenários de eventos.

2.2 As Dimensões do Desacoplamento

A eficácia de um *middleware* moderno pode ser avaliada, em grande medida, pela sua capacidade de garantir o desacoplamento entre os componentes que ele interliga. O conceito de desacoplamento não é monolítico — ele se manifesta em pelo menos três dimensões distintas, cada uma com implicações técnicas e operacionais próprias.

1. **Desacoplamento Espacial.** Os produtores de dados não precisam conhecer a identidade, o endereço de rede ou sequer a quantidade de consumidores interessados na informação que eles publicam. Toda a responsabilidade de manter o catálogo de assinantes e de resolver as rotas de entrega recai sobre a infraestrutura de *middleware*. Na prática, isso significa que um novo consumidor pode ser adicionado ao ecossistema sem que o produtor seja sequer notificado ou modificado, o que facilita a extensão do sistema.
2. **Desacoplamento Temporal.** As entidades participantes não precisam estar ativas ao mesmo tempo. Em um acoplamento síncrono convencional, a comunicação entre um produtor P e um consumidor C exige que ambos estejam operando simultaneamente no instante t . Formalmente, o estado de acoplamento $S_{coupling}$ requer:

$$S_{coupling} = P(t) \cap C(t) \neq \emptyset \quad (1)$$

Com a introdução de um intermediador Q (a fila ou o barramento do *middleware*), a interação é segmentada em dois momentos independentes:

$$P(t_1) \rightarrow Q \quad \text{e} \quad Q \rightarrow C(t_2) \quad \text{onde} \quad t_1 \neq t_2 \quad (2)$$

O resultado prático é que uma manutenção programada no consumidor C no instante t_1 não afeta o produtor P : as mensagens ficam retidas no intermediador e são entregues quando C volta a estar disponível. Esse isolamento temporal é particularmente relevante em ambientes que operam em fusos horários distintos ou que possuem janelas de manutenção concorrentes.

3. Desacoplamento de Sincronização. A submissão de dados é projetada para ser não-bloqueante do ponto de vista do emissor. O produtor publica a mensagem (ou o evento) e retoma imediatamente suas operações, sem aguardar confirmação de recebimento por parte do consumidor. O *middleware* assume o encargo de garantir a entrega, geralmente combinando persistência em disco, confirmações internas (*acks*) e mecanismos de retentativa. Do ponto de vista do código do produtor, a operação de publicação se comporta como uma chamada local rápida, mesmo que a entrega real ocorra segundos ou minutos depois.

Essas três dimensões, quando combinadas, produzem um ecossistema em que os componentes operam de forma genuinamente autônoma. Um serviço pode ser implantado, escalado ou substituído sem impactar os demais — desde que a interface dos eventos (o contrato do *schema*) seja respeitada.

2.3 Integração Corporativa e Padrões Arquiteturais

O roteamento e a filtragem implementados pelos *middlewares* modernos não são improvisações ad hoc; eles se apoiam em catálogos de padrões consolidados, como os *Enterprise Integration Patterns* (EIP) documentados por Hohpe e Woolf [3]. Entre esses padrões, o *Content-Based Router* (Roteador Baseado em Conteúdo) é particularmente relevante para as plataformas orientadas a eventos. O funcionamento é direto: quando uma mensagem chega ao *middleware*, a plataforma inspeciona o conteúdo do registro submetido — por exemplo, campos específicos de um documento JSON — e avalia predicados lógicos para definir quais consumidores devem recebê-la. Em vez de cada consumidor receber todas as mensagens e filtrar localmente as que lhe interessam, o roteamento acontece no próprio barramento, economizando recursos computacionais nas pontas.

Plataformas como o Amazon EventBridge implementam esse conceito de forma nativa. O EventBridge funciona como um barramento central de eventos dentro do ecossistema AWS, absorvendo responsabilidades que, em arquiteturas anteriores, precisariam ser codificadas manualmente em cada aplicação. Três exemplos

concretos dessas responsabilidades são:

- **Retentativas com recuo exponencial (*exponential backoff*):** Quando a entrega de um evento falha, o EventBridge não tenta reenviá-lo imediatamente. Em vez disso, ele espaça as tentativas de forma crescente — primeiro 1 segundo, depois 2, depois 4, e assim por diante — para evitar sobrecarregar um consumidor que já está com problemas.
- **Adição de variação aleatória (*jitter*):** Às retentativas com recuo exponencial, o EventBridge adiciona um componente aleatório ao intervalo de espera. Sem esse recurso, múltiplos produtores que falhassem ao mesmo tempo sincronizariam suas retentativas, criando picos de carga periódicos no consumidor. O *jitter* distribui essas retentativas de forma mais uniforme ao longo do tempo.
- **Filas de mensagens mortas (*Dead Letter Queues* — DLQ):** Eventos que não podem ser entregues após o esgotamento das tentativas de retentativa são encaminhados para uma fila separada. Essa fila serve como um repositório de diagnóstico: a equipe de operações pode inspecionar os eventos não entregues, identificar a causa da falha e, se necessário, reprocessá-los manualmente.

Essas funcionalidades, geridas no nível do *middleware*, aliviam o código das aplicações de negócio de uma quantidade considerável de lógica de infraestrutura. Em um cenário sem *middleware*, cada serviço precisaria implementar seus próprios mecanismos de retentativa, *circuit breaking* e registro de falhas — o que não só multiplicaria o esforço de desenvolvimento, mas também introduziria inconsistências operacionais entre os diferentes serviços do ecossistema.

3 Comparações de Arquiteturas e Soluções de Mensageria

As decisões arquiteturais que envolvem a adoção de EDA em detrimento de serviços síncronos REST — e a escolha da tecnologia específica de *middleware* — não podem ser tomadas apenas com base em intuição ou preferência pessoal. Elas

requerem uma análise que leve em conta dados empíricos, métricas estruturais do código e características operacionais mensuráveis. Esta seção apresenta essa análise em duas frentes: primeiro, a comparação entre os estilos arquiteturais REST e EDA sob a ótica da modularidade; depois, o confronto entre as principais plataformas de *middleware* disponíveis no mercado.

3.1 Arquitetura Orientada a Eventos (EDA) *vs.* Estilo REST

Lazzari e Farias [4] conduziram um estudo empírico em que implementaram um mesmo sistema de gestão de pedidos sob ambas as abordagens — REST e EDA — e submeteram as duas implementações a cinco cenários evolutivos, medindo um conjunto de métricas estruturais a cada ciclo. Os resultados dessa investigação oferecem uma visão bastante concreta das vantagens e das desvantagens de cada modelo.

3.1.1 Separação de Interesses e Acoplamento

A primeira constatação do estudo diz respeito à separação de interesses (*Separation of Concerns* — SoC). Na implementação orientada a eventos, a introdução de novas funcionalidades afetou, em média, 40,74% menos módulos do que na implementação REST. Essa métrica, chamada CDC (*Concern Diffusion over Components*), mede quantos componentes do sistema precisam ser modificados para acomodar uma nova funcionalidade. Um CDC menor indica que cada funcionalidade está mais encapsulada, com fronteiras mais bem definidas entre os módulos.

Na mesma direção, as transições de interesse medidas em linhas de código (CDLOC — *Concern Diffusion over Lines of Code*) foram 48,71% menores na abordagem EDA. Esses números corroboram a premissa teórica de que, quando os componentes se comunicam por eventos em vez de chamadas diretas, as preocupações de cada módulo tendem a se misturar menos.

Entretanto, os dados revelam um fenômeno que, à primeira vista, parece contraditório. As métricas de dependência direta apontam na direção oposta: a dependência interna (*Dep_In*) cresceu 575% e a dependência externa (*Dep_Out*) aumentou 237,5% no cenário EDA em relação ao REST. Essa aparente contradição tem uma

explicação técnica precisa. Embora os módulos de negócio não se modifiquem simultaneamente — ou seja, uma alteração no serviço de pagamentos não obriga uma alteração no serviço de estoque —, a infraestrutura de eventos introduz uma forte dependência de definições estruturais compartilhadas: os *schemas* dos eventos, as interfaces de serialização, os contratos de tópico. Esses artefatos funcionam como pontos de convergência que todos os módulos precisam referenciar. O desacoplamento em tempo de execução, portanto, não elimina o acoplamento em tempo de compilação ou de definição de contrato; ele apenas o desloca para uma camada diferente.

3.1.2 Coesão, Complexidade e Tamanho do Código

Uma expectativa comum quando se discute EDA é a de que a orientação a eventos simplificaria o código, tornando-o mais enxuto e mais fácil de manter. Os dados do estudo de Lazzari e Farias refutam parcialmente essa expectativa. A coesão relacional de interfaces (H) — que mede o grau em que os métodos de uma classe estão relacionados entre si — foi 26,48% mais favorável na implementação EDA. Isso significa que, individualmente, cada classe tende a ser mais focada e coesa quando o sistema adota eventos.

O problema aparece nas métricas de complexidade e tamanho. A complexidade estrutural (R) disparou 572,09% na implementação EDA. Essa escalada não é um artefato estatístico; ela reflete a quantidade de código auxiliar que a orientação a eventos exige. Para cada evento no sistema, é necessário definir *schemas* de serialização, implementar adaptadores de entrada e saída, registrar assinaturas em tópicos, configurar controladores de deserialização e, frequentemente, manter classes utilitárias para transformação de dados entre formatos internos e externos. Cada uma dessas peças é, isoladamente, simples. O volume acumulado, porém, é significativo.

Os números confirmam essa tendência: o total de linhas lógicas de código (LOC) cresceu 76,52% na implementação EDA, e o número de atributos instanciados aumentou 70,37%. A arquitetura REST, em contraste, mostrou-se mais compacta, gerando um volume menor de código e um esforço de compreensão inicial mais reduzido. Para projetos com equipes pequenas ou prazos apertados, esse fator não é

negligenciável.

Esses resultados sugerem que a escolha entre REST e EDA não é uma questão de superioridade absoluta. A orientação a eventos oferece ganhos concretos de modularidade e isolamento, mas cobra esse benefício na forma de uma infraestrutura mais volumosa e de uma curva de aprendizado mais íngreme. A decisão depende do perfil do projeto: sistemas que evoluem frequentemente e que precisam de alta resiliência tendem a se beneficiar do EDA; sistemas mais estáveis, com fluxos bem definidos e equipes reduzidas, podem encontrar no REST uma solução mais pragmática.

3.2 Comparativo Multidimensional de Plataformas de Middleware

A escolha da plataforma de *middleware* afeta diretamente os limites operacionais do sistema distribuído. Não basta decidir entre REST e EDA; é preciso escolher qual tecnologia concreta vai implementar o barramento de eventos ou a fila de mensagens. E essa escolha envolve *trade-offs* que nem sempre são intuitivos.

Para subsidiar essa decisão, diversos estudos de *benchmarking* — entre eles os trabalhos de Gopisetty [5] e Arafat, Tasmin e Poudel [6] — avaliaram as principais tecnologias do mercado sob condições controladas de carga. Os resultados desses estudos permitem construir um mapa comparativo que considera não apenas o desempenho bruto, mas também a arquitetura subjacente e o custo humano de operação.

A Tabela 1 resume a arquitetura e o modelo de comunicação de cada tecnologia avaliada.

Tabela 1: Modelos de Comunicação e Arquitetura de Middlewares

Tecnologia	Modelo de Comunicação	Arquitetura Base	Escalabilidade e Confiabilidade
Apache Kafka	Log Distribuído (<i>Pull</i>)	<i>Cluster</i> com armazenamento em disco	Massiva horizontal, alta persistência
Apache Pulsar	Multi-tenant Separado	Brokers independentes do <i>storage</i>	Horizontal com <i>geo-replicação</i> nativa
RabbitMQ	<i>Broker</i> de Filas AMQP (<i>Push-to-Pull</i>)	Servidores gerenciando filas em disco/RAM	Escala vertical, garantias de entrega (<i>acks</i>)
EventBridge	Barramento de Eventos (<i>Push-to-Push</i>)	<i>Serverless</i> multi-tenant na nuvem AWS	Elástica sob demanda, com retentativas e DLQ
Amazon SQS	<i>Message Queue</i> (<i>Pull</i>)	<i>Serverless</i> de fila ponto a ponto	Praticamente ilimitada, até 14 dias de retenção
GCP Pub/Sub	Tópicos Pub/Sub Global	<i>Serverless</i> integrado (Google Cloud)	Global automática, persistência distribuída
Redis Streams	Memória otimizada	<i>Cluster</i> in-memory	Latência ultrabaixa com retenção efêmera

Cada linha da tabela representa uma filosofia diferente de engenharia. O Apache

Kafka, por exemplo, trata cada tópico como um log imutável e sequencial, armazenado em disco. Os consumidores leem esse log por operações de *pull*, controlando o ritmo de consumo de forma independente. Isso confere ao Kafka uma persistência robusta e uma capacidade de reprocessamento que outras plataformas não oferecem nativamente — mas em troca exige que a equipe gerencie partições, réplicas e rebalanceamentos.

O Apache Pulsar adota uma abordagem diferente: separa a camada de *broker* (roteamento de mensagens) da camada de armazenamento (Apache BookKeeper). Essa separação arquitetural facilita a escalabilidade independente de cada camada, mas introduz uma complexidade operacional adicional, pois são dois sistemas distribuídos que precisam ser gerenciados em conjunto.

Na outra ponta do espectro, plataformas *serverless* como o Amazon EventBridge e o Google Cloud Pub/Sub abstraem completamente a infraestrutura. Não há *cluster* para provisionar, partições para dimensionar ou discos para monitorar. A contrapartida é que o provedor de nuvem gerencia tudo de forma opaca, em um ambiente multi-inquilino compartilhado, o que limita o controle sobre parâmetros de desempenho e introduz variabilidade na latência.

A Tabela 2 apresenta os dados empíricos de desempenho obtidos nos estudos de *benchmarking* referenciados.

Tabela 2: Benchmarking Operacional e Parâmetros Físicos Sob Carga de Teste

Framework	Vazão Máxima (msg/seg)	Latência p95 (ms)	Complexidade (FTE)	Limitação Chave
Apache Kafka	~ 1.200.000	18	2,3	Rebalanceamento frio, particionamento estático
Apache Pulsar	~ 950.000	22	1,8	Complexidade arquitetural (<i>BookKeeper</i>)
NATS JetStream	~ 800.000	15	1,2	Limitações no tamanho da mensagem e memória
Redis Streams	~ 650.000	8	0,8	Retenção ligada à capacidade de memória RAM
RabbitMQ	~ 450.000	32	1,5	Teto de <i>throughput</i> , alto <i>overhead</i> de rotas
GCP Pub/Sub	~ 300.000	78	0,4	Oscilações de latência regional
EventBridge	~ 200.000	85	0,3	Piso de latência rígido, controle de taxa

3.2.1 Análise dos Resultados de Desempenho

Os números da Tabela 2 revelam uma separação clara entre duas famílias de tecnologias. De um lado, as plataformas autogerenciadas — Apache Kafka, Apache Pulsar, NATS JetStream — entregam vazões na casa das centenas de milhares a mais

de um milhão de mensagens por segundo, com latências em percentil 95 que ficam abaixo dos 25 milissegundos. Do outro, as plataformas *serverless* — GCP Pub/Sub e EventBridge — operam com vazões significativamente menores e latências que ultrapassam os 75 milissegundos.

Essa diferença de desempenho bruto, contudo, não conta a história toda. A coluna de complexidade operacional, medida em FTE (*Full-Time Equivalent* — equivalentes em tempo integral de engenheiro), inverte a equação. O Apache Kafka, com sua vazão de 1,2 milhão de mensagens por segundo, demanda o equivalente a 2,3 engenheiros dedicados exclusivamente à manutenção da infraestrutura: gerenciamento de partições, monitoramento de réplicas, ajuste de configurações de rebalanceamento, aplicação de patches de segurança. O EventBridge, em comparação, requer 0,3 FTE — praticamente um terço de um engenheiro.

Para uma empresa que precisa processar liquidações financeiras em tempo real, com requisitos de ordenamento transacional rigoroso e propriedades ACID, a escolha pelo Kafka ou pelo Pulsar é quase inevitável. A capacidade de processar mais de um milhão de mensagens por segundo com latência controlada e log imutável não tem equivalente nas plataformas *serverless* atuais. Nesse tipo de cenário, o custo humano de operação é justificado pela criticidade da aplicação.

Em cenários diferentes — integrações entre microsserviços SaaS, coreografia de processos de negócio, notificações entre domínios —, a equação muda. A latência de 85 milissegundos do EventBridge é perfeitamente aceitável quando os consumidores não são sensíveis a variações na casa das dezenas de milissegundos. E a economia operacional é substancial: não há discos para monitorar, não há *clusters* para redimensionar, e as *Dead Letter Queues* são gerenciadas automaticamente.

O Redis Streams ocupa um nicho particular nesse espectro. Com uma latência em p95 de apenas 8 milissegundos e um FTE de 0,8, ele combina baixa latência com custo operacional moderado. A limitação é estrutural: toda a retenção depende da capacidade de memória RAM disponível, o que torna a plataforma inadequada para cenários que exigem reproprocessamento de grandes volumes de dados históricos.

3.2.2 O *Trade-off* Central

Os dados dos estudos de *benchmarking* [5, 6] convergem para uma conclusão que precisa ser enunciada com clareza: não existe uma plataforma de *middleware* que seja simultaneamente a mais rápida, a mais flexível, a mais econômica e a mais simples de operar. A escolha da tecnologia é, por natureza, um exercício de *trade-off*.

Esse *trade-off* pode ser mapeado ao longo de dois eixos principais. O primeiro eixo opõe desempenho bruto (latência e vazão) a custo operacional (FTE e complexidade de gestão). O segundo eixo opõe controle granular (configuração de partições, políticas de replicação, ajuste fino de parâmetros) a conveniência gerenciada (abstração da infraestrutura, escalabilidade elástica, zero provisionamento). Cada projeto, com seus requisitos específicos de volume, latência, orçamento e tamanho da equipe, cairá em um ponto diferente desse mapa — e a escolha da plataforma deve refletir essa posição.

4 Conclusão

A análise desenvolvida ao longo deste relatório permite formular algumas conclusões fundamentadas sobre a relação entre estilos arquiteturais, *middleware* e os requisitos operacionais de sistemas distribuídos modernos.

A primeira conclusão diz respeito à trajetória de migração dos modelos síncronos para os modelos assíncronos. Essa migração não é uma questão de tendência ou de modismo — ela responde a limitações técnicas concretas. Quando a carga transacional cresce e os serviços se multiplicam, o modelo síncrono baseado em REST atinge um teto de escalabilidade que não pode ser contornado sem mudanças arquiteturais profundas. O bloqueio de *threads* imposto pelas chamadas síncronas limita de forma determinística a capacidade de resposta horizontal do sistema, e a propagação em cascata de falhas compromete a disponibilidade do ecossistema como um todo.

A segunda conclusão refere-se ao impacto da orientação a eventos sobre a modularidade do código. Os dados empíricos de Lazzari e Farias [4] demonstram que a EDA reduz a difusão de interesses entre módulos: funcionalidades novas afetam menos componentes, e as linhas de código dedicadas a cada preocupação ficam mais con-

centradas. Essa é uma vantagem genuína para sistemas que evoluem com frequência e que precisam suportar equipes de desenvolvimento independentes. O custo, como os mesmos dados demonstram, é um aumento expressivo na complexidade estrutural. O volume de código auxiliar — *schemas*, serializadores, configuradores — pode dobrar ou triplicar em relação a uma implementação REST equivalente. Para equipes que adotam EDA, essa complexidade precisa ser gerenciada com disciplina, sob o risco de criar um sistema que, embora modular em teoria, torna-se opaco na prática.

A terceira conclusão trata da escolha da plataforma de *middleware*. Os dados de *benchmarking* [5, 6] mostram que a decisão entre plataformas autogerenciadas e *serverless* é governada por um *trade-off* entre desempenho e custo operacional. Em cenários de alto volume e baixa latência — liquidações financeiras, telemetria industrial, análise de dados em tempo real —, plataformas como o Apache Kafka oferecem capacidades que as soluções *serverless* não conseguem equiparar no momento atual. Em cenários de integração corporativa com requisitos de latência mais flexíveis, plataformas como o Amazon EventBridge e o GCP Pub/Sub oferecem uma relação custo-benefício mais favorável, ao eliminar quase integralmente o esforço de gestão de infraestrutura.

Nenhum *framework* resolve todos os problemas de latência, tolerância a falhas e modularidade simultaneamente. A arquitetura ótima para um dado sistema resulta da ponderação cuidadosa entre o isolamento no código que o padrão arquitetural proporciona e a sustentação do rendimento que a infraestrutura escolhida viabiliza. Essa ponderação não é genérica; ela precisa ser feita caso a caso, com base nos requisitos concretos do projeto e nos recursos — humanos e computacionais — efetivamente disponíveis.

Referências Bibliográficas

- [1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Distributed Systems: Principles and Paradigms*. 1. ed. Nova Jérsei: Prentice Hall, 2002. ISBN 9780130888938.
- [2] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. *Distributed Systems: Concepts and Design*. 5. ed. Boston: Addison-Wesley, 2012. ISBN 9780132143011.
- [3] HOHPE, Gregor; WOOLF, Bobby. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. 1. ed. Boston: Addison-Wesley, 2003. ISBN 9780321200686.
- [4] LAZZARI, Luan; FARIAS, Kleinner. Event-driven Architecture and REST Architectural Style: An Exploratory Study on Modularity. *Computación y Sistemas*, Cidade do México, v. 27, n. 3, 2023. Disponível em: https://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1665-64232023000300338.
- [5] GOPISETTY, Satyanarayana. Exactly-Once, Always Auditable: Benchmarking the Latency, Throughput, and Evidential Integrity Trade-offs of AWS Serverless Orchestration versus Choreography for High-Frequency Payment Settlements. *International Journal of Computer Technology (IACSE-IJCT)*, v. 7, n. 1, p. 14–36, 2026. DOI: 10.5281/zenodo.20266481.
- [6] ARAFAT, Jahidul; TASMIN, Fariha; POUDEL, Sanjaya. Next-Generation Event-Driven Architectures: Performance, Scalability, and Intelligent Orchestration Across Messaging Frameworks. *arXiv preprint arXiv:2510.04404*, 2025. Disponível em: <https://arxiv.org/pdf/2510.04404>.
- [7] ALAASAM, A. B.; RADCHENKO, G.; TCHERNYKH, A. Stateful stream processing for digital twins: Microservice-based Kafka stream DSL. In: *International Multi-Conference on Engineering, Computer and Information Sciences (SIBIRCON)*. Novosibirsk: IEEE, 2019. p. 804–809.

- [8] AMAZON WEB SERVICES. *What Is Amazon EventBridge?*. AWS Documentation, 2019. Disponível em: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>.